

Serving landing pages as compiled static HTML

How */r/code* stops booting a React application for every visitor arriving from a paid ad, and starts serving a pre-built HTML document with the tracking pixel already running in the first few hundred bytes.

7 of 7 LIVE PAGES COVERED IN V1 six block types, measured	~4.6 KB TYPICAL PAGE OVER THE WIRE brrotli, estimated	~0.2 s UNTIL THE PIXEL FIRES versus ~9 s on slow 4G	5 PROBLEMS FOUND IN THE CODE two would break the first deploy
--	--	--	--

This document explains a change to how influencer landing pages are served. It is written to be read end to end before any code is written, because three of the five problems it describes are invisible until the change is live and expensive to discover then.

WHAT THIS DOCUMENT IS NOT

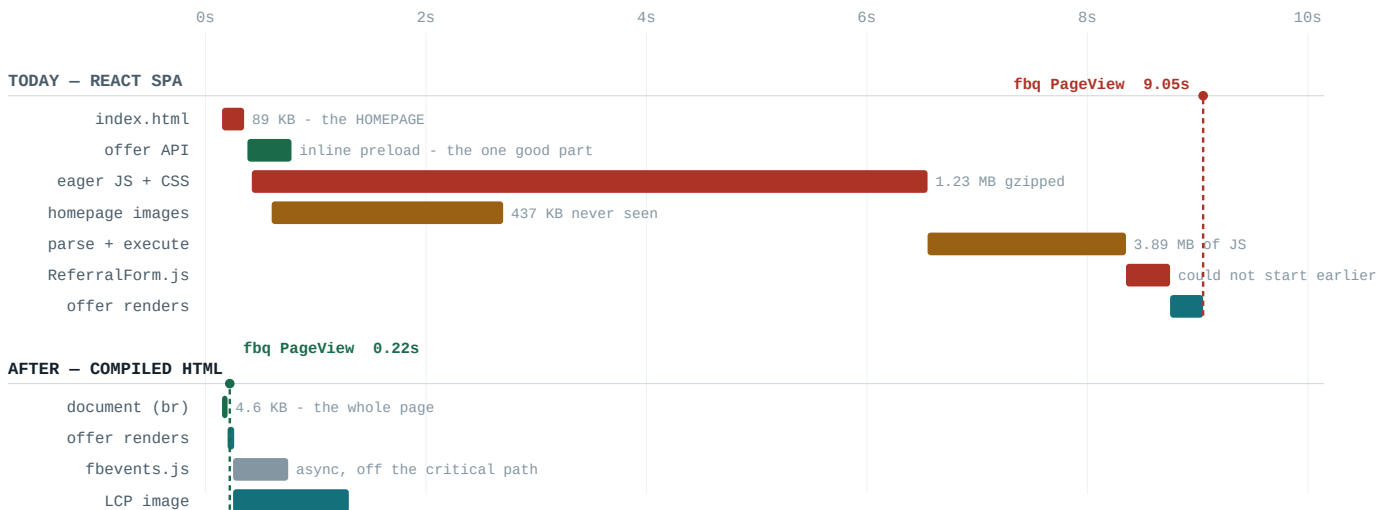
It contains no conversion-rate predictions. An earlier draft of this plan claimed a jump from "28–33%" to "85–92%" landing page view rate and a "+250% more visitors" result. Those numbers were not measured and could not be — they are removed. What is measurable is **bytes over the wire** and **time until the pixel fires**, and only those are claimed here. Whether that converts better is something your own lead counts will tell you within a week of the rollout.

CONTENTS	
1 · What happens today, in order	The load waterfall, measured
2 · What the change actually is	Compile on save, serve from Express
3 · Why an Express route alone does nothing	nginx owns /r/ today
4 · Five problems found by reading the code	Two would break the first deploy
5 · Losing React means losing escaping	The security layer this needs
6 · How pages stay fresh	The cache and version strategy
7 · What survives, what degrades, what is lost	Tracking and analytics
8 · The work, in order	A scope correction, then phases
9 · How you will know it worked	Verification and the kill switch

SECTION 1

What happens today, in order

A visitor taps your ad on Facebook. Their phone opens `https://sub.silacod.com/r/NEO-ARTIZEN`. nginx finds no file at that path and falls back to `frontend/dist/index.html` — which, since prerendering was added, is **the fully rendered SILACOD marketing homepage**. So the first thing the visitor's browser paints is your homepage, complete with navigation, hero and 23 images, before React wipes it off the screen and starts again. Everything below has to finish **before your Meta Pixel can record that the visit happened at all**:

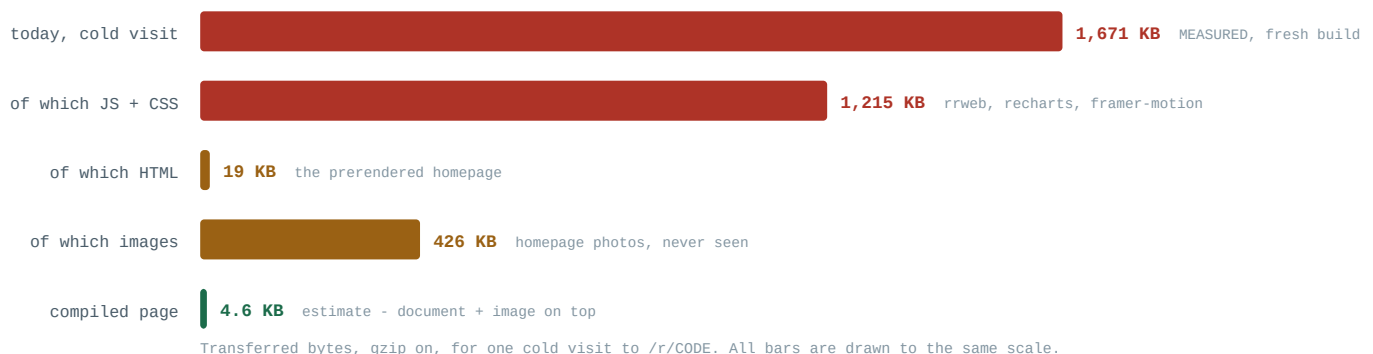


Byte counts are MEASURED from `frontend/dist` (built 14 Aug). Timings are modelled on those bytes at Lighthouse 'Slow 4G' — 1.6 Mbps, 150 ms round trip — on a mid-tier Android device. The two dashed markers are the claim: today the pixel cannot fire until 3.89 MB of JavaScript has downloaded, parsed and executed; after the change it fires while the document is still being parsed.

Why the pixel is the number that matters

Everything else on that chart is a performance concern. The pixel is a **money** concern. Meta optimises your ad delivery from the events it receives. A visitor who taps, waits, and leaves after two seconds is a visitor Meta never hears about — so it keeps buying you more of that same audience. You pay for the click either way.

Moving `fbq('init')` and `fbq('track', 'PageView')` into the `<head>` of a document that is already complete when it arrives means the event fires during HTML parsing — before any image, before any stylesheet, and with no JavaScript framework involved at all.



Re-measured on 16 Aug from a fresh build of current master: 5,282,886 bytes of eager JavaScript and CSS, 1,244,821 gzipped. That is slightly WORSE than the 14 Aug build (5,196,951 / 1,226,154) — the problem has grown, not shrunk. The original sketch's '5.2 MB' figure was accurate. `rrweb` is still statically imported by `LiveStreamInspector.tsx:5` and `AbandonedCarts.tsx:4`, and `recharts` by the three dashboards, so all of it loads on a landing page that uses none of it.

Why splitting the bundle did not help here

`vite.config.ts` already splits vendor code into named chunks, with careful comments explaining why. The split is real — but **separate chunks are not the same as lazy chunks**. A chunk is only deferred if nothing on the startup path imports it, and `App.tsx` statically imports the dashboard and admin screens that pull in charts and session replay. So they are separate files that are all still downloaded:

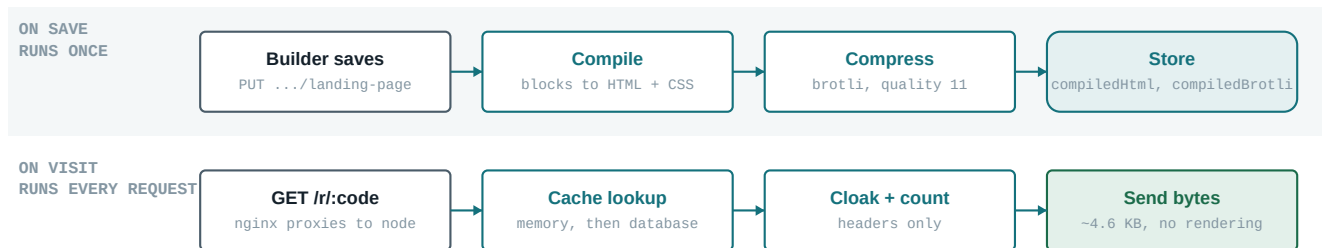
CHUNK	RAW BYTES	GZIPPED	WHY IT LOADS ON A LANDING PAGE
<code>assets/index-*.js</code>	3,976,085	910,796	entry chunk — around 130 pages statically imported
<code>assets/index-*.css</code>	328,902	45,563	render-blocking
<code>vendor-charts</code>	433,927	114,388	recharts — dashboards, statically imported in <code>App.tsx</code>
<code>vendor-rrweb</code>	262,456	81,818	session replay — admin screens, statically imported
<code>vendor-react</code>	165,216	53,750	react + router
<code>vendor-motion</code>	114,377	37,554	framer-motion
<code>vendor-rrweb.css</code>	1,923	952	render-blocking
eager total	5,282,886	1,244,821	downloaded on every page, /r/ included
<code>ReferralForm-*.js</code>	~28,000	~9,200	the only chunk that is actually the landing page

The landing page's own code is the last row: **27,976 bytes**. Everything above it is overhead that a visitor who will only ever see one offer page pays in full — and cannot begin to pay until the entry chunk has executed, because the lazy import lives inside it.

SECTION 2

What the change actually is

The landing page builder already stores every page as JSON — a list of blocks in `ReferralLinkLandingPage.customStructure`. Today that JSON is shipped to the browser and turned into a page by React, on the visitor's phone, every single time. The change is to do that work **once, on the server, when the page is saved**, and store the resulting HTML.



The expensive half runs when an influencer clicks Save. The visitor half reads a cache and writes bytes to a socket.

The part that makes this maintainable

Compiled output goes stale — that is the usual objection to this approach, and the usual answer is a backfill script somebody has to remember to run. This design avoids that entirely with a **version number**.

```
// backend/src/services/landingCompiler/index.ts
export const COMPILER_VERSION = 1;

// on read:
if (row.compiledHtml && row.compilerVersion === COMPILER_VERSION) {
  return row.compiledHtml; // reuse
}
return await compileLanding(link); // recompile and persist
```

Fix a bug in the compiler, bump the number to 2, deploy. Every stored page now fails the equality check, so the first visitor to each one pays a single recompile and everybody after that gets the fixed version. **No migration, no backfill, no cron job.** The fix propagates itself, page by page, driven by real traffic — which also means your busiest pages are fixed first.

Two independent switches control whether a visitor sees compiled HTML.

`SSG_LANDING` is an environment variable on the server: off, shadow, or on. `landingPage.ssgEnabled` is a per-link column. Both must say yes.

You chose to deploy with `SSG_LANDING=on` and `ssgEnabled` defaulting to true, so every link switches on deploy. Keeping the environment variable means you can revert the entire feature to the React page with `pm2 restart` — no code change, no rebuild, about fifteen seconds.

SECTION 3

Why an Express route alone does nothing

This is the single most important correction to the original sketch, and it is the difference between the feature working and it appearing to do nothing at all.

Express does not currently serve any HTML. It handles `/api/v1/*` and `/uploads/*`, and nothing else. The React application — including `/r/:code` — is served directly by nginx from disk. Adding `app.get('/r/:code', ...)` to the backend today would produce a handler that is never called, because the request never reaches Node.

```
# /etc/nginx/sites-available/silacod    (generated by setup.sh)

location / {
    root    /var/www/silacod/frontend/dist;
    index  index.html;
    try_files $uri $uri/ /index.html;    # <-- /r/CODE lands here
}

location /api/ {
    proxy_pass http://127.0.0.1:3001;
}
```

A new location block takes over that path. nginx matches prefix locations by **longest match, not by file order**, so `/r/` wins over `/` wherever it is written. The `^~` is still worth adding — it stops nginx evaluating regex locations once this prefix has matched:

```
# nginx matches prefix locations LONGEST-FIRST, not in file order, so `^~`
# beats `/` wherever this block sits. `^~` is still worth having: it stops
# nginx evaluating regex locations once this prefix has matched.

location ^~ /r/ {
    limit_req zone=landing burst=40 nodelay;
    proxy_pass http://127.0.0.1:3001;
    proxy_http_version 1.1;

    # $host, not $http_host: Host lowercased with the port stripped, which is
    # what getSubdomainFromRequest() compares against in production.
    proxy_set_header Host      $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;

    gzip off;                # the body is already brotli-compressed upstream
    proxy_read_timeout 15s;
}
```

This block lives in `/etc/nginx/sites-available/silacod` on the VPS. `setup.sh` generates that file, but only at provisioning time — it does not run on deploy. **The change must be made in `setup.sh` for the next rebuild AND applied by hand to the live server**, followed by `nginx -t && systemctl reload nginx`. Forgetting the second half is the most likely way for this project to appear to have no effect whatsoever.

Something the build already produces and nobody uses

While tracing this, one thing turned up that is worth fixing on its own merits.

`frontend/scripts/prerender.mjs` runs after every build and snapshots your marketing pages. In doing so it **overwrites** `dist/index.html` **with the prerendered homepage** — 89,297 bytes in the current build. It also writes a pristine, unrendered shell to `dist/spa.html`, 7,818 bytes, for exactly this situation.

But nginx's `try_files` names `/index.html`. So `spa.html` is generated on every single deploy and **read by nothing**, while every visitor to a landing page downloads your homepage markup and watches React throw it away.

This means the *fallback* path improves too. When the compiler declines a page — unsupported block, compile error, feature switched off — the Express handler serves `spa.html` rather than `index.html`. Same React application, same behaviour, about **11× less markup**. Pages that never get compiled still get faster.

Where the route sits inside Express

Once nginx forwards the request, position in the middleware stack matters — some of what runs on API calls is pure latency on a document request:

MIDDLEWARE	RUNS FOR /R/?	WHY IT IS PLACED THERE
<code>helmet</code>	yes	Sets the default Content-Security-Policy. Free on JSON; on HTML it blocks inline scripts and <code>connect.facebook.net</code> . The route must strip and replace it.
<code>cors</code>	yes	Synchronous origin check. A top-level navigation sends no Origin, so this is a no-op.
<code>compression</code>	yes	Hooks <code>res.end</code> . Kept upstream of the route so clients that refuse brotli still get gzip.
<code>express.json / urlencoded</code>	yes	Skipped entirely for a GET with no body.
<code>morgan + requestLogger</code>	yes	One log line, plus a fire-and-forget TrafficLog insert after the response finishes. Deliberately kept — compiled pages should keep feeding traffic analytics exactly as the SPA did.
<code>securityHeaders</code>	yes	Sets X-Frame-Options: DENY. The route relaxes this to SAMEORIGIN for <code>/r/</code> only.
<code>scanner blocklist</code>	yes	Seven substring tests for <code>/.env</code> , <code>/wp-admin</code> and friends. Effectively free.
<code>ipFilter</code>	yes	Async. Awaits a settings lookup cached for 30 s. Kept upstream on purpose — a blocked IP should not be served a page.
<code>sanitizeInput</code>	no	Async. Awaits the same cached settings, then XSS-scrubs body, query and params — all empty on this route. Pure latency, so the route mounts <i>above</i> it.
<code>maintenance</code>	no	Async. Returns a JSON 503 for every path during maintenance. Ads keep running during maintenance and a JSON error is not a page, so landing pages stay up.
<code>globalRateLimiter</code>	no	Scoped to <code>/api/v1</code> and stays there. 100 req/min would be actively harmful — bursty ad traffic is the point of this route. nginx rate-limits it instead.
<code>notFoundHandler</code>	no	What <code>/r/</code> would hit today if nginx forwarded it: a JSON 404. This is why the nginx change and the Express route must land together.

The route mounts immediately after `ipFilter` — late enough that security and logging still apply, early enough to skip two asynchronous settings lookups and the API rate limiter that a document request has no business paying for.

SECTION 4

Five problems found by reading the code

None of these are speculative — each was confirmed against the actual file. The first two would break the deploy outright. The third is a trap that is currently harmless for an accidental reason. The last two are bugs that already exist and which this project would otherwise inherit and disguise.

BLOCKER 1 backend/src/utils/subdomain.ts:3

Every visitor arriving from an ad would get a 404

The existing host check, `validateInfluencerSubdomain()`, decides which hostname a request came from using `getRequestHost()` — which prefers the `Origin` header, then `Referer`, and only falls back to `Host` if neither is present.

That order is correct for the API call the React app makes, where the request is same-origin. It is wrong for a **document** request. When somebody taps your ad, the browser sends no `Origin` header at all and sets `Referer` to the ad network. So the check reads the host as `facebook.com`, finds it does not match the influencer's subdomain, and rejects the request.

```
// getRequestHost(), simplified — the order is the problem
if (originHeader) requestHost = new URL(originHeader).host;
if (!requestHost && refererHeader) requestHost = new URL(refererHeader).host;
if (!requestHost && hostHeader) requestHost = hostHeader;

// Top-level navigation from an ad:
// Origin: (absent)
// Referer: https://www.facebook.com/    <-- this wins
// Host: sub.silacod.com                 <-- never reached
```

If missed: 100% of paid traffic sees a 404 while direct visits and your own testing work perfectly. The fix is a separate `validateInfluencerHost()` that binds on `Host` only.

BLOCKER 2 backend/src/index.ts:182

Helmet's default policy would block every tracking pixel

`helmet` is mounted with only a `crossOriginResourcePolicy` override, which means its default `Content-Security-Policy` is active — including `script-src 'self'`.

This is harmless today because Express only ever returns JSON, and a policy on a JSON response affects nothing. The moment Express returns HTML, that same header starts being enforced by the browser against the page: inline scripts blocked, `connect.facebook.net` blocked, YouTube frames blocked.

If missed: Pages render but no pixel fires — the exact failure this project exists to fix, introduced by the fix itself. Worse, it also applies on the SPA fallback path, which nginx currently serves with no CSP at all. **Both handlers must strip the header** and set their own.

BLOCKER 3 frontend/vite.config.ts:19

Returning visitors would keep seeing the old page

The site registers a service worker through vite-plugin-pwa. In its default generate mode the worker answers **navigation** requests from its own cache, and no navigateFallbackDenylist is configured. So `/r/CODE` would be served the cached React shell from the visitor's own device — the request never reaching nginx, let alone Node.

It is worth being precise about this one, because the currently deployed build makes it look harmless. **That build has no sw.js in it**, so no worker is installed and nothing is intercepted today. But a fresh build run on 16 August **does** produce one — 2,198 bytes, registering a navigation route, with no denylist anywhere in it. The absence is an accident of a stale build, not the configuration.

```
# fresh `vite build`, 16 Aug:
sw.js                2,198 bytes  <- generated
workbox-1ef09536.js  15,082 bytes

$ grep -c 'NavigationRoute' sw.js -> 1    answers every navigation
$ grep -c 'denylist'      sw.js -> 0    nothing excluded

// frontend/vite.config.ts - must ship in the SAME deploy
workbox: {
  maximumFileSizeToCacheInBytes: 8000000,
  navigateFallbackDenylist: [/^\/r\/$/],
}
```

If missed: The compiled page works perfectly in a private window and not at all for anyone who has visited the site before — a genuinely hard bug to diagnose from a report of "it didn't change for me". One line, same deploy.

BLOCKER 4 BlockRendererer.tsx:844 and :979

A contained bug that this change would turn into a serious one

The slider block builds a `<style>` element by string interpolation and injects it with `dangerouslySetInnerHTML`. Four values are interpolated: `block.id`, `cardGap`, `marqueeSpeed` and a shadow colour. All four come from the saved JSON, and the save route performs **no validation of any kind** — no schema, no type check, no length limit. It destructures `req.body` and hands it to Prisma five lines later.

Today the impact is limited to CSS. Assigning markup to a `<style>` element's `innerHTML` does not create elements — the browser treats the contents as stylesheet text. So an injected `<img onerror=>` creates nothing and runs nothing. It is a real bug, but a contained one: restyling the page, or pulling a background image from an external server.

The same bytes written into server-generated HTML source do execute. That is the difference this project introduces.

```
// frontend/src/components/helper/sitebuilder/BlockRendererer.tsx:843
<style dangerouslySetInnerHTML={{ __html: `
  @keyframes marquee-${id} {           // id comes from saved JSON,
    ...                               // which is never validated
  }
  .track-${id} { gap: ${cardGap}px; }
`} />
```

If missed: The compiler must derive every class and keyframe name from the block's **index**, never from `block.id`, and every colour and length must pass an allow-list before reaching CSS. **The slider has never been placed on any page** (see section 8), so the compiler will not touch this code for the foreseeable future — but the save route validation in phase 0 should still land, because it closes the underlying hole for the app that exists today.

BLOCKER 5 `lead.routes.ts:143`

Pack pricing is already silently wrong in one case

When a visitor picks a product from a `products` block, the checkout form sends `productVariant` as a composite string like `"Sac cuir (Pack 2)"`. The server-side `getPackPrice()` matches `productVariant` against the bare pack name — `"Pack 2"`. The composite string can never match, so those leads silently fall back to the product's retail price.

If missed: This is a cash-collection bug that exists today. The compiler must reproduce the current behaviour **exactly**, including this quirk, and the bug should be fixed in its own change — otherwise a pricing correction and a rendering rewrite land together, and any change in revenue is impossible to attribute.

SECTION 5

Losing React means losing escaping

This section is the one worth reading twice. It describes a class of bug that does not announce itself in testing.

Right now, when the builder stores a heading of `<script>alert(1)</script>`, React renders those characters as visible text. React escapes everything it interpolates, automatically, with no way to switch it off by accident. That protection is not something the codebase does — it is something the framework does *for* the codebase.

Building HTML with string templates on the server removes that protection entirely. Every value now has to be escaped by hand, correctly, for the specific context it lands in — and the same value needs different treatment depending on where it goes:

WHERE THE VALUE LANDS	EXAMPLE FIELD	WHAT BREAKS IT	HANDLED BY
HTML text	<code>text.text</code> , <code>hero.title</code>	<code><script></code> in the value becomes a real script tag in the document.	escape <code>&</code> <code><</code> <code>></code> <code>"</code> <code>'</code> <code>`</code>
HTML attribute	<code>image.alt</code> , <code>image.url</code>	A stray double quote closes the attribute early; everything after it becomes new attributes, e.g. <code>onerror=</code> .	same escape, always
CSS value	<code>themeColor</code> , every <code>*Color</code>	Inside <code><style></code> there is no attribute boundary to escape. A value can close the rule, or fetch a tracking URL via <code>url()</code> .	allow-list + fallback
URL	<code>button.link</code> , <code>video.url</code>	<code>javascript:</code> in an href runs on click. Browsers ignore tabs and NUL bytes inside the scheme, so naive checks miss it.	scheme allow-list
JS string	<code>pack names</code> , <code>pixel ids</code>	<code></script></code> anywhere inside a JSON string ends the script block, whatever the JSON escaping did.	escape <code><</code> <code>></code> <code>&</code> <code>U+2028/9</code>

Why the xss package already installed is not the answer

— **It runs at the wrong layer.** `sanitizeInput` cleans `req.body` on the way in, and it is gated behind a database setting an administrator can switch off. Whether your output is safe must not depend on a runtime toggle — and rows already in the database were written under whatever the setting happened to be then.

— **It works on the wrong unit.** `xss()` is an HTML sanitiser: it decides which *tags* survive. It leaves quotes and ampersands in ordinary text alone. A stored alt-text of `" onerror=alert(1) x="` contains no tag at all, so `xss()` never looks at it — but dropped into `alt="..."` it escapes the attribute.

— **It would change what your pages render.** `xss()` *permits* a whitelist of tags. A text block containing `<a href=x>` shows as literal text today; passed through `xss()` it would become a working link. Silently changing what influencers' pages display is not an acceptable side effect of an escaping decision.

The one subtle case worth understanding

Most of the escaping is mechanical. This one is not, and it is the reason `safeurl()` strips characters before testing rather than testing the raw string:

```
// Both of these NAVIGATE in a real browser:
<a href='java[TAB]script:alert(1)'\>  with a real tab character
<a href='java[NUL]script:alert(1)'\>  with a real NUL byte

// So the check STRIPS control characters first, then tests:
const probe = v.replace(/[\u0000-\u0020...]/g, '').toLowerCase();
if (/^(javascript|vbscript|file|blob|about):/.test(probe)) return '';
```

A naive `startsWith('javascript:')` check passes both of those straight through. This is exactly the kind of detail that makes hand-rolled escaping risky, and the reason the escape layer is a small, tested module with its own unit tests rather than helpers scattered across the block renderers.

SECTION 6

How pages stay fresh

Storing rendered output creates one obvious risk: showing somebody a page that no longer reflects reality. There are three layers, and it is worth knowing which one catches which kind of change.

LAYER	LIVES FOR	CATCHES
Memory cache per Node process	10 minutes	Nothing on its own — it is purely a speed layer. Bounded by both an entry count and a byte ceiling so one unusually large page cannot push the process past its 500 MB restart limit.
Explicit invalidation on write	immediate	An influencer saving the page; pixels being added or removed; a subdomain or custom domain changing. These fire a targeted purge, so the next visitor recompiles.
Version stamp COMPILER_VERSION	until bumped	Every change you make to the compiler itself. Bump the constant and all stored output is invalid at once, with no backfill.

Three scenarios, end to end

- **An influencer edits their page.** The save route writes `customStructure`, compiles immediately, stores the HTML, and purges the memory entry. The next visitor gets the new page. Time to propagate: instant.
- **You fix a bug in the compiler.** Bump `COMPILER_VERSION` and deploy. Nothing is recompiled at deploy time — instead the first visitor to each page triggers one recompile, and everyone after gets the fix. Your busiest pages heal first, and a page nobody visits costs nothing.
- **A product price changes.** Nothing explicitly purges the landing page, because the price lives on a different table. The ten-minute freshness window catches it. This is the deliberate trade: a short staleness window in exchange for not having to find and hook every write path that could possibly affect a page.

Why compiled pages are not cached by browsers or a CDN.

The response carries `Cache-Control: no-store`. That looks like a wasted opportunity until you notice that the same URL must return *different things to different people* — the cloaking rules redirect some visitors and not others — and that every view has to count as a click. A shared cache would serve one visitor's cloaking verdict to the next, and would hide visits from your click counter.

The cost of that decision is small: the server is doing a memory lookup and a socket write, on the order of a millisecond or two.

SECTION 7

What survives, what degrades, what is lost

Bypassing the React application means every side-effect that happened *because* React was running has to be accounted for. Most can be preserved; several actually improve; two turn out to have been broken for a while.

SIDE-EFFECT	STATUS	WHAT IT TAKES
ReferralLink.clicks + click rows	LOST	Today this is a side-effect of the offer API call. The SSG handler must call the same code. The 10-second de-duplication map has to be SHARED, or a visitor whose browser also hits the API gets counted twice.
100-click notification	RIDES ALONG	Lives inside the click increment, so preserving one preserves the other.
Bot / direct / desktop cloaking	UPGRADED	Decided from User-Agent and Referer before the first byte. Catches bare crawlers that never ran the JavaScript check. Fix the regex first — see the warning below.
Country / IPv6 / IP-range cloaking	UPGRADED	Uses the real socket IP and <code>geoip-lite</code> , already a dependency. Removes a third-party round trip to <code>ipapi.co</code> on the critical path, and the visitor can no longer block the check.
Language cloaking	NEEDS CARE	<code>Accept-Language</code> is a weighted list, not <code>navigator.language</code> . Parse it and test the top entry, or verdicts shift for bilingual visitors.
VPN / ASN cloaking	LOST server-side	<code>geoip-lite</code> has no VPN or network-operator fields. Keep these as a small inline re-check, or accept the loss.
Right-click blocking	CLIENT-SIDE	Emitted as a ~180-byte inline script, only when the setting is on.
Abandoned-cart capture	ALREADY DEAD	Not caused by this project. The socket is refused on <code>/r/</code> paths today, so no rows are being written at all. Restoring it is a separate improvement.
rrweb session replay	ALREADY DEAD	Same cause. Nothing is being lost here that is not already gone.
Traffic log for the visit	UPGRADED	The document request itself gets logged with the real path, instead of the API call standing in for it.
x-client-* telemetry	DEGRADED	Screen size, window size and similar arrive from JavaScript today. A plain document request has none. Inline a few bytes to re-post them, or accept lower fidelity.
Maintenance-mode behaviour	FIXED	<code>MaintenanceGuard</code> already documents an intent to keep <code>/r/</code> up during maintenance. Mounting the route above the middleware finally makes that true.
WhatsApp click tracking	MUST BE PORTED	The widget's tracking call has to be emitted into the static page or the counter stops moving.
Pixels and lead submission	MUST BE PORTED	The entire point of the page. Both are rebuilt in the compiled runtime.

A CORRECTION WORTH MAKING EXPLICITLY

When we discussed this, you asked to preserve abandoned-cart capture, on the understanding that serving static HTML would break it. **That premise was wrong, and the mistake was mine.**

`SocketContext.tsx:19` returns early for any path beginning `/r/` and never opens a socket connection. So on landing pages socket is null, the `checkout:progress` emitter at `ReferralForm.tsx:69` returns immediately, and `socket?.emit('checkout:complete')` at line 356 is a no-op. **No abandoned-cart rows are being recorded for landing pages today**, and the same applies to session replay.

This changes the decision. Rebuilding the capture as a small beacon is still worth doing — but it is a *repair of an existing bug*, not a dependency of this project, and it should be scheduled separately. Before doing the work, check whether any `CheckoutAttempt` rows have appeared for referral traffic recently; if the table is dry, that confirms it.

SECTION 8

The work, in order

SCOPE CORRECTION — AND A CAVEAT ON IT

The original plan picked seven block types to support first. Those were chosen from the builder's palette — **nobody checked what real pages contain**. Querying the database changes the picture completely: only **six** block types have ever been placed on a page, four of the seven planned types are used by nobody, and **video** — one of the most common — was left out.

The caveat: those counts come from the local development database, which backend/.env points at (localhost:5432). Its pages are named aaaa, dddd, ccccaa — scratch data. The figures below are therefore **illustrative of the method, not of your business**.

Before choosing a scope, run `scripts/landing-block-usage.ts` on the VPS against the production database. It prints exactly this table for real pages.

BLOCK TYPE	TIMES PLACED	PAGES	STATUS	WHAT IT NEEDS
express_checkout	5	5	V1	The form itself. Cannot be static — needs a runtime.
video	5	4	V1	Player with unmute and play overlays.
button	4	3	V1	Sticky behaviour and entrance animations.
image	2	2	V1	The only pure-markup block in real use.
whatsapp	2	2	THEN	Floating widget, open/close, click tracking. Takes 3/7 to 5/7.
audio	2	2	THEN	Play/pause, seek, speed control. Takes 5/7 to 7/7.
header	—	—	NOT USED	Never placed on any page.
hero	—	—	NOT USED	Never placed on any page.
text	—	—	NOT USED	Never placed on any page.
spacer	—	—	NOT USED	Never placed on any page.
slider	—	—	NOT USED	Never placed — note this is where the CSS bug lives.
countdown	—	—	NOT USED	Never placed on any page.
products	—	—	NOT USED	Never placed. Its API endpoint does not exist.

The practical effect is a **harder** phase 1, not an easier one. Of the six blocks in real use, only **image** is pure markup — the other five all need a small amount of JavaScript. The work shifts away from reproducing CSS and towards writing compact runtimes. That is worth knowing before committing to a timeline.

Ordered so that nothing user-facing changes until the groundwork is in place, and so the first visible step is also the smallest.

0	Groundwork NO VISIBLE CHANGE	Extract the click-recording code so one implementation serves both paths — sharing its de-duplication map is what prevents double counting. Add validation to the save route, which currently stores any JSON up to 10 MB without inspection. Narrow the builder's read query so it stops returning the new HTML column. Add the service-worker denylist.
1	The compiler and the route ALL 7 LIVE PAGES	The six block types actually in use, the escape layer with its own tests, the cache, the Express route, the nginx block, server-side cloaking, and the checkout form rebuilt in plain JavaScript. Any page using a block outside the six is declined and served the React version instead — a refusal, not a failure.
2	Only when needed DEFERRED	The seven remaining block types have never been placed on a page. Build one when a page actually uses it; until then the fallback covers it correctly. Spending effort here before that happens is building for nobody — which is precisely the mistake the original scope made.
3	Tuning MEASURED IMPROVEMENT	Image dimensions read at compile time to eliminate layout shift; the largest image given priority; pre-connections opened only for pixel providers actually in use.

Where the effort actually goes

Before the database was checked, the answer looked like CSS: roughly two hundred utility classes — `shadow-xl`, `text-4xl`, `md:text-5xl`, `backdrop-blur-sm`, responsive grids — reproduced by hand until the spacing matches. That work is still there, but it is no longer the bulk of it.

The real cost is the runtimes. Five of the six blocks in use need JavaScript: the checkout form with its validation and pixel firing, a video player with unmute and play overlays, an audio player with seek and speed control, the floating WhatsApp widget, and the sticky button. Each has to be written in plain JavaScript, kept small enough to inline, and behave exactly as the React version does — including the existing quirks, so that no behaviour changes underneath the rollout.

This is the part to be honest about in any estimate. It is more work than the original plan implied, and the reason is that the original plan chose the easy blocks without checking whether anyone used them.

SECTION 9

How you will know it worked

Immediately after deploying

Run this against a real link. Everything you need to know is in the headers:

```
curl -sI https://sub.silacod.com/r/YOURCODE

# Expected:
#   HTTP/2 200
#   content-type: text/html; charset=utf-8
#   content-encoding: br
#   cache-control: no-store
#   content-security-policy: default-src 'none'; ...

# The failure to look for: TWO content-security-policy headers, or one
# containing `script-src 'self'`. That is helmet's default leaking
# through, and it means no pixel on the page will fire.
```

Then check these four things, in this order

- **Real sizes, not estimates.** `curl -s URL | wc -c` against `curl -s -H 'Accept-Encoding: identity' URL | wc -c`. The estimates in this document should be replaced with whatever these actually say.
- **The page in a browser that has visited the site before.** Not a private window — a normal one, ideally on a phone that has used the platform. This is the check that would catch a service worker interfering, if the PWA is ever repaired.
- **Meta Events Manager.** PageView events should appear promptly and in greater numbers than before, because visitors who used to leave during the load are now being counted. Lead events should match your Lead row count.
- **Compile failures.** `SELECT count(*) FROM referral_link_landing_pages WHERE "compileError" IS NOT NULL;` — anything above zero is a real bug, since an unsupported block is recorded as a decline rather than an error.

The measurement that actually decides it

Not Lighthouse. **Leads per link per day, compared against the same links the week before.**

Page speed is the mechanism, not the goal. If leads do not move, the speed improvement was real and the bottleneck was somewhere else — the offer, the audience, the price — and you will have learned something more useful than a performance score. Watch `ReferralLink.clicks` alongside it: if clicks hold steady while leads rise, the change did what it was meant to do.

If something is wrong

```
# Revert everything to the React page. No rebuild, no code change.
pm2 set silacod-api:SSG_LANDING off # or edit the env file
pm2 restart silacod-api --update-env

# Or turn off a single link, leaving the rest compiled:
#   UPDATE referral_link_landing_pages SET "ssgEnabled" = false
#   WHERE "referralLinkId" = <id>;

# Or, for one request only, to compare side by side in a browser:
#   https://sub.silacod.com/r/YOURCODE?__ssg=0
```

The nginx change stays in place either way — with `SSG_LANDING=off` the Express route simply serves the React application itself, using the small `spa.html` shell rather than the 89 KB homepage. So even the fully reverted state is better than today.

FOUND ALONG THE WAY — SEPARATE FROM THIS WORK

Three live bugs turned up while reading the code for this plan. None are caused by this project and none should be fixed inside it, but the first one is costing money right now.

— **The bot filter redirects every human visitor.** `cloaking.ts:16` contains a bare `moz` pattern, and every browser sends `Mozilla/5.0`. Any link with bot filtering enabled and no explicit allow-list has been sending 100% of its paid traffic to `wikipedia.org`. `instagram` and `whatsapp` are also bare patterns, so in-app browsers are caught too. **Fix this before moving cloaking to the server.**

— **The PWA does nothing but cost 89 KB per visit.** The build emits a registration script but no service worker, so every visitor requests `/sw.js`, receives the homepage, and fails to register.

— **Three image URLs point at localhost.** The prerendered homepage contains `http://localhost:4173/uploads/...` baked in at build time; those images are broken for every visitor.